Prepared by Sumaiya Tarannum Noor

Date: 12 11 2026

# Lecture 3/4 - GPU Architecture

1. Why are GPUs more suitable than CPUs for massively parallel computations, even though individual GPU threads may execute more slowly?

Ans-

GPUs and CPUs are designed with different goals in mind.

While a CPU is designed to excel at executing a serial sequence of operations (called thread) as fast as possible and can execute a few tens of these threads in parallel.

A GPU is designed to excel at executing thousands of threads in parallel, trading off lower single-thread performance to achieve much greater total throughput.

GPUs are specialized for highly parallel computations and devote more transistors to data processing unit. CPUs dedicate more transistors to data caching and flow control.

GPU maximize throughput by hiding latency through massive parallelism rather than minimizing individual thread latency.

It prioratizes massive computation completion ~~rather~~ through multidimensional parallelism ~~rather than indi~~ thread parallelism rather than individual or a few threads performance.

2. How does the design philosophy of a CPU differ from ~~that~~ that of a GPU in handling execution latency?

Ans. A CPU is designed to execute a small number of threads as quickly as possible. It dedicates substantial hardware resources to minimize stalls through: large caches, out-of-order executions, branch prediction and high clock speed.

GPU is designed to ~~excel~~ execute far more threads than it has processing cores. Rather than maximizing ~~individ~~ individual thread performance, it achieves high

throughput through massive parallelism.

It has thousands of arithmetic units and simplified control logic. It has large register files.
Unlike CPUs GPUs generally avoid complex branch prediction and out-of-order execution, sharing instruction-control overhead across threads through SIMT.
Each SM maintains the execution state of many resident threads, enabling rapid switching between warp.

When one warp stalls on a high-latency memory operation, the scheduler can immediately issue instructions from another ready warp.

GPUs maximize throughput by hiding latency through massive parallelism rather than individual thread latency.

Sa Su Mo Tu We Th Fr Date:

3. What are the three fundamental abstraction of CUDA Programming model, and why are they important?

Ans- there are three key abstraction in the CUDA Programming model:

(i) Hierarchy of thread groups: Programs are executed in threads but can make reference to groups of threads in a nearest hierarchy, from blocks to grid.

(ii) Hierarchy of memories: Thread groups at each level of the hierarchy have access to a memory resource for communication within the group.

(iii) Barrier Synchronization: Thread groups can coordinate execution by means of ~~barriers~~ barriers.

These CUDA programming model abstractions are important because they enable programs to scale across different GPU architectures.

Q. What is the difference between host code and device code in CUDA? How do they interact?

Ans- In CUDA, host code runs on host (CPU) while device code runs on device (GPU)

Because the CPU and GPU are separate hardware units with their own hardware units with their own memory ~~access~~ spaces, CUDA uses heterogenous programming model where CPU acts as the "manager" and GPU acts as the parallel "workhouse".

Host code and device code communicate through a structured workflow managed by the CPU using CPU runtime API.

Q. Why does CUDA organize GPU execution into a hierarchy of grids, thread blocks and threads?

The host code, which is code that runs on the CPU, can use CUDA APIs

to copy data between the host memory and device memory, start code executing on the GPU, and wait for data copies or GPU code to complete.
The CPU and GPU can both be executing code simultaneously.

5. Why does CUDA organize GPU execution into a hierarchy of grids, thread blocks and blocks?

Ans. CUDA organizes execution into a hierachy of threads, thread blocks and grids to bridge the gap between software programming models and the physical hardware of GPU, enabling massive parallelism.
Threads ~~executes its~~ are the basic execution unit. It executes it's own instructions.
Thread blocks are groups of threads that execut together on a single Streaming Multiprocessor (SM).
Grids of are collections of thread blocks that span the entire GPU utilizing all available SMs.

Sa Su Mo Tu We Th Fr Date:

6. Why must CUDA thread blocks be independent of one another for scalable execution?

Ans. CUDA thread blocks must be independent of one another so that the GPU can schedule, execute and scale them in any order — in parallel, sequentially, or across an arbitary number of hardware cores without risking dead locks or race condition.

7. What is the difference between PTX and SASS, and why does CUDA use both representations?

Ans. PTX (Parallel Thread Execution): It is NVIDIA's virtual instruction set architecture (ISA) and intermediate representation. It is abstract and architecture agnostic.

SASS (Streaming Assembly): It is NVIDIA's native, hardware-specific machine code

in human-readable assembly language.
It runs directly on the physical GPU cores.
It is architecture dependent.

CUDA uses both to balance hardware specific performance and future-proof compatability.

PTX has forward compatability as same PTX code can run for various generations of GPU.

SASS provides peak hardware performance as it allows the compiler to optimize instructions, register allocation and scheduling for a specific GPU architecture. ~~without any risk~~

8. How does PTX enable forward compatability across different generations of NVIDIA GPUs?

Ans. PTX (Parallel Thread Execution) enables forward compatability across different generations of NVIDIA GPUs by acting as a virtual instruction set architecture (ISA) and intermediate representation (IR) rather than fixed physical machine code. PTX can be JIT compiled (↳ Just-in-time) for newer supported GPUs.

9. Why does CUDA provide each thread with a unique index, even though all threads execute the same kernel code?

Ans- CUDA provides each thread with a different index so that thousands of threads running the exact same kernel code can be executed on different set of data simultaneously.

10. Why are CUDA kernel launches generally asynchronous with respect to CPU execution?

Ans- CUDA kernel launches are asynchronous with respect to CPU to allow the CPU (host) and GPU (device) to execute tasks concurrently, maximizing overall hardware utilization and hiding latency.

11. How does CUDA's SIMT execution model differ from traditional SIMD execution?

Ans -
The difference between SIMT (Single Instruction, multiple thread) and SIMD (Single Instruction, Multiple Data are shown below;

SIMD (AVX-512): One vector instruction operates on multiple data elements, with vector width explicitly exposed.

SIMT (CUDA): Programmers write ~~code~~ scalar code for individual threads, while hardware organizes them into 32-thread warps.

SIMT combines the programmability of independent threads with the hardware efficiency of coordinated, warp-level execution

12. What is the role of a Streaming Multiprocessor (SM), and why is it the fundamental unit of GPU performance optimization?

Ans. Streaming Multiprocessor (SM) is the main execution unit for CUDA kernels.

It contain warp schedulers, register file, shared memory / L1 cache, CUDA cores, tensor cores and other execution units.
It's main role is to execute many warp (groups of threads) in parallel.

It is the fundamental unit of GPU performance optimization because SM schedules warps and executes instructions from resident thread blocks. Therefore, the amount of computation which is the main performance parameter for GPU, depends of SM.

13. Why must all threads of a CUDA thread block execute on the same SM?

Ans. All threads of a CUDA thread block must execute on the same Streaming Multiprocessor (SM) to enable fast on-chip communication, shared memory access, and hardware-level synchronization.

Sa Su Mo Tu We Th Fr Date:

14. How does a warp scheduler hide memory latency without saving and restoring thread execution state?

Ans - When one warp stalls, the scheduler can execute instructions from other ready warps.

It Provides a larger pool of ~~executable resident warps~~ executable work.

Occupancy = Resident warp / Maximum supported warps

proportion.

Sufficient occupancy helps hide ~~latency~~ memory and instruction latency, keeping the execution units busy.

occupancy ~~is the~~ measures the proportion of an SM's maximum supported warps and that are currently resident.

15. Why does warp divergence reduce execution efficiency even when all 32 threads are actively executing a kernel?

Ans - Warp divergence occurs when threads within the same warp follow different control-flow paths.

Warp divergence reduces SIMT efficiency because different execution paths cannot fully utilize all 32 thread lanes simultaneously. It can also approach 32x loss in extreme cases.

16. Why can two different warps follow different branches without causing warp divergence?

Ans - ~~Two~~ ~~different~~ Warp divergence only happens ~~when~~ within a single warp, never between different warps.

Each of 32 threads warps is scheduled independently and has its own program counter, so two separate warps can take completely different branches simultaneously without causing warp divergence.

17. How does independent thread scheduling improve execution flexibility, and why does it not eliminate warp divergence?

Ans. In independent thread scheduling, each thread maintains its own program counter and execution state, allowing more flexible scheduling of divergent threads.

Independent thread scheduling helps prevent certain dead locks. The use of each threads own program counter and execution state improve execution flexibility.

But, it does not eliminate warp divergence, because a warp still shares a single instruction fetch and decode unit. So, it cannot execute two different instruction simultaneously.

18. How does predication differ from traditional branching, and why is it particularly useful for short conditional operations?

Ans. Predication executes instructions from all possible conditional paths ~~(ed)~~ and only commits the results of the valid paths, whereas traditional branching alters the program's control flow by jumping to a single chosen path.
Predication is ideal for small conditional ~~blocks (typically just a few instructions)~~ operations because executing a few extra instructions takes less time than recovering from a branch misprediction.

19. What is a GPU occupancy, and how does it influence the ability of an SM to hide latency?

Ans. GPU Occupancy measures the proportion of an SM's maximum supported warps and that are currently resident.

Occupancy = Resident warps / maximum Supported warps

When one warp stalls, the scheduler can execute instructions from other ready warps. Higher occupancy provides a larger pool of ~~executable work~~ resident warps, increasing the likelihood that executable work is available.
Sufficient occupancy helps hiding memory and instruction latency by keeping execution units busy.

21. Why does achieving 100% occupancy not necessarily guarantee maximum GPU performance?

Ans. High occupancy improves latency by hiding but does not necessarily maximize performance.

Higher occupancy may require fewer registers per thread, potentially causing spills to slower local memory.

Additional occupancy provides little benefit once execution or memory pipelines are saturated.

For these reasons, 100% occupancy does not necessarily guarantee maximum GPU performance.

20. Why can increasing register usage per thread reduce the number of concurrent warps on SM?

Ans. Each Streaming Multiprocessor (SM) has a fixed size physical register file shared by all active threads.

When each thread uses more registers, the SM runs out of register to allocate for new warps. As running warps are all occupying all the registers, the SM cannot allocate register for new warp.

~~28.~~

22. How do Thread-Level Parallelism (TLP) and Instruction-Level Parallelism (ILP) differ in their approaches to latency hiding?

Ans. TLP (Accross multiple threads): Hides latency by ~~context switching~~ switching between warps. when a warp is stalled on a heavy memory operation, the scheduler executes instruction from another ready warp or thread.

ILP (within a single thread): Hides latency by reordering and overlapping independent instructions from a single execution stream. when an instruction stalls the hardware looks ahead in the same instruction stream to find other independent instructions to execute immediately.

23. Why can increasing shared-memory usage per thread block reduce the number of blocks executing concurrently?

Ans. Shared memories in an SM has limited size.

$$\text{Maximum resident blocks} = \text{floor}\left(\frac{\text{Available Shared Memory}}{\text{Shared memory per block}}\right)$$

So, if the resident thread blocks are using shared memory massively, the SM won't be able to allocate shared memory for new block, which reduces the number of blocks executing concurrently.

24. Why do Tensor cores achieve significantly higher matrix multiplication throughput than conventional CUDA cores?

Ans. Tensor cores are dedicated hardware units designed to accelerate matrix multiply-accumulate (MMA) operations.

A conventional FP32 CUDA core can perform one fused multiply add (FMA), equivalent to 2 FLOPs, per cycle at peak throughput. Tensor cores perform many such operations across matrix tiles.

Tensor cores achieve substantially higher (theoratically 15 x) arithmetic throughput by using specialized matrix processing hardware rather than general-purpose scalar arithmetic units.

25. Why does lower-precision arithmetic such as BF16 or FP8, improve Tensor core throughput?

Ans — BF16, FP8 improve Tensor core performance because they require ~~smaller~~ smaller, more energy-efficient arithmetic circuits. It enables greater computational density.

26. Why is keeping Tensor cores supplied with data often more challenging than performing the actual matrix computations?

Ans— Keeping Tensor cores supplied with data is more challenging than performing the math because modern GPU computation speed have outpaced memory bandwidth, creating a hardware bottleneck known as "memory wall".

27. How does shared memory differ from L1 cache, and why is shared memory useful for tiled matrix multiplication?

Ans— Shared memory provides fast, explicitly managed storage for cooperating threads within a thread block.

Unlike L1 cache, which is hardware managed, shared-memory contents are explicitly controlled by the programmer.

Tiled matrix multiplication reduces global memory traffic by reusing operands.

Shared memory is useful for tile matrix multiplication, because in tiling strategy, a data is load from HBM to shared memory.

28. Why do coalesced global memory accesses improve GPU memory bandwidth utilization, while shared-memory bank conflicts reduce performance?

Ans— Coalesced Global memory access maximize the amount of useful data transferred per memory transaction.

~~So, it improves GPU~~

Global memory requests are grouped into memory transactions to maximize bandwidth utilization.

Shared-memory is organized into 32 banks to support parallel access by threads within a warp.

Accessing different banks enables efficient parallel operations.
Multiple thread accessing the different address same bank at the same time creates bank conflict. ~~Therefore~~
Bank conflicts may serialize access and reduce throughput, thus reduces performance.

29. Why can memory bandwidth become the primary performance battleneck even when a GPU has substantial computational capacity?

Ans. GPU computational throughput has increased much faster than memory bandwidth.
HBM bandwidth has not increased at the same rate.
Modern GPU computation speed has outpaced memory bandwidth, creating a hardware battleneck known as "memory wall". So, even if a GPU has substantial computational power it's performance wouldn't reach it's full potential.